

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO11 – Develop Program Using Functions

Nominal Duration: Theory 3.5 Hours / Practical 6 Hours

L.O 11: DEVELOP PROGRAM USING FUNCTIONS

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Explain the concept and purpose of Python functions.
2. Identify common types of Python functions.
3. Create user-defined functions using def.
4. Call and execute functions.
5. Distinguish parameters from arguments.
6. Pass values to functions.
7. Pass lists to functions.
8. Apply positional arguments.
9. Apply default parameter values.
10. Apply keyword arguments.
11. Apply arbitrary positional arguments using *args.
12. Apply arbitrary keyword arguments using **kwargs.
13. Return values from functions using return.
14. Explain local and global variable scope.
15. Apply the pass statement in incomplete function definitions.
16. Develop recursive functions using appropriate base conditions.
17. Create and execute lambda functions.
18. Apply functions for code reuse and modular program development.
19. Test and debug Python functions.

INFORMATION SHEET No. 11.1

Title: Understanding and Using Python Functions

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain Python functions and their purposes.
- Identify function types.
- Explain function syntax and structure.
- Create user-defined functions.
- Call functions.
- Explain parameters and arguments.
- Pass values and lists into functions.
- Use positional, default, keyword, and arbitrary arguments.
- Apply return.
- Explain local and global scope.
- Apply pass.
- Explain recursion and base cases.
- Create lambda functions.
- Explain code reusability and modular programming.
- Identify common function errors.

1. Introduction to Python Functions

A function is a named block of reusable code designed to perform a specific task.

Instead of repeating the same code many times, the code can be placed inside a function and executed whenever required.

Example without a function:

```
print("Welcome to Python")
print("Welcome to Python")
print("Welcome to Python")
```

Using a function:

```
def welcome():
    print("Welcome to Python")
```

```
welcome()
welcome()
welcome()
```

Output:

```
Welcome to Python
Welcome to Python
Welcome to Python
```

2. Why Functions Are Used

Functions help programmers to:

- Reduce repeated code.
- Organize programs.
- Divide large problems into smaller tasks.
- Improve readability.
- Simplify testing.
- Simplify debugging.
- Reuse existing logic.
- Support modular programming.
- Maintain programs more easily.

3. Basic Function Structure

A user-defined function is created using the `def` keyword.

Syntax:

```
def function_name():  
    statements
```

Example:

```
def greet():  
    print("Hello Python")
```

The function is defined, but it does not execute until it is called.

4. Function Definition

Example:

```
def display_course():  
    print("AI Basic Python Developer")
```

Components:

Component	Description
def	Keyword used to define a function
display_course	Function name
()	Contains parameters if required
:	Ends the function header
Indented block	Function body

5. Function Naming Rules

Function names should:

- Begin with a letter or underscore.
- Not begin with a number.
- Not contain spaces.
- Not use Python keywords.
- Use meaningful names.
- Prefer lowercase words separated by underscores.

Recommended:

```
def calculate_total():  
    pass
```

Less clear:

```
def ct():  
    pass
```

6. Calling a Function

A function is executed using a function call.

Example:

```
def greet():  
    print("Hello")
```

```
greet()
```

Output:

Hello

The statement:

```
greet()
```

is the function call.

7. Calling a Function Multiple Times

Example:

```
def welcome():  
    print("Welcome")
```

```
welcome()  
welcome()  
welcome()
```

A function may be reused as many times as required.

8. Types of Functions

At this level, Python functions can be grouped into:

1. Built-in functions
2. User-defined functions
3. Anonymous or lambda functions
4. Recursive functions

9. Built-in Functions

Built-in functions are already provided by Python.

Examples:

```
print()
len()
type()
range()
input()
sum()
max()
min()
```

Example:

```
scores = [80, 90, 75]

print(len(scores))
print(sum(scores))
```

10. User-Defined Functions

A user-defined function is created by the programmer.

Example:

```
def show_name():
    print("Dara")

show_name()
```

11. Lambda Functions

A lambda function is a small anonymous function created using the `lambda` keyword.

Example:

```
double = lambda x: x * 2

print(double(5))
```

Output:

10

Lambda functions are explained in more detail later in this Information Sheet.

12. Recursive Functions

A recursive function is a function that calls itself.

Example:

```
def countdown(number):  
    if number == 0:  
        print("Done")  
    else:  
        print(number)  
        countdown(number - 1)
```

countdown(3)

Output:

3
2
1
Done

13. Parameters

A parameter is a variable listed in a function definition.

Example:

```
def greet(name):  
    print("Hello", name)
```

Here:

name

is a parameter.

14. Arguments

An argument is an actual value supplied when the function is called.

Example:

greet("Dara")

Here:

"Dara"

is an argument.

Therefore:

Parameter → variable in function definition

Argument → actual value supplied during function call

15. Passing One Value to a Function

Example:

```
def display_name(name):  
    print("Name:", name)
```

```
display_name("Dara")
```

Output:

Name: Dara

16. Passing Multiple Values

Example:

```
def student_info(name, age):  
    print("Name:", name)  
    print("Age:", age)
```

```
student_info("Dara", 20)
```

Output:

Name: Dara

Age: 20

17. Positional Arguments

Positional arguments are matched according to their positions.

Example:

```
def student_info(name, age):  
    print("Name:", name)  
    print("Age:", age)
```

```
student_info("Dara", 20)
```

Python matches:

"Dara" → name
20 → age

Changing the order changes the meaning.

18. Passing a List to a Function

A list can be passed into a function.

Example:

```
def display_students(students):  
    for student in students:  
        print(student)
```

```
names = ["Dara", "Sokha", "Vanna"]
```

```
display_students(names)
```

Output:

```
Dara  
Sokha  
Vanna
```

19. Processing a List in a Function

Example:

```
def calculate_total(scores):  
    total = 0
```

```
    for score in scores:  
        total += score
```

```
    print("Total:", total)
```

```
student_scores = [80, 90, 75]
```

```
calculate_total(student_scores)
```

Output:

```
Total: 245
```

20. The return Statement

The return statement sends a result back to the function caller.

Example:

```
def add(a, b):  
    return a + b
```

```
result = add(5, 10)
```

```
print(result)
```

Output:

15

21. print() vs return

Consider:

```
def add(a, b):  
    print(a + b)
```

This displays a value.

Compare with:

```
def add(a, b):  
    return a + b
```

This sends the value back so it can be stored or used elsewhere.

Example:

```
result = add(5, 10)
```

```
print(result * 2)
```

Output:

30

22. Returning Multiple Values

A function can return multiple values.

Example:

```
def calculate(a, b):  
    addition = a + b  
    subtraction = a - b  
  
    return addition, subtraction  
  
result1, result2 = calculate(10, 5)  
  
print(result1)  
print(result2)
```

Output:

```
15  
5
```

Python returns the values as a tuple.

23. Default Parameter Values

A default parameter has a predefined value.

Example:

```
def greet(name="Student"):  
    print("Hello", name)  
  
greet()  
greet("Dara")
```

Output:

```
Hello Student  
Hello Dara
```

If no argument is supplied, Python uses the default value.

24. Multiple Parameters with Default Values

Example:

```
def student_info(name, course="Python"):  
    print("Name:", name)  
    print("Course:", course)  
  
student_info("Dara")
```

Output:

Name: Dara
Course: Python

Override the default:

```
student_info("Dara", "Machine Learning")
```

25. Default Parameter Rule

Required parameters should normally appear before parameters with default values.

Correct:

```
def student(name, course="Python"):
    print(name, course)
```

Incorrect:

```
def student(course="Python", name):
```

This causes a syntax error.

26. Keyword Arguments

Keyword arguments identify parameters by name during a function call.

Example:

```
def student_info(name, age, course):
    print("Name:", name)
    print("Age:", age)
    print("Course:", course)
```

```
student_info(
    course="Python",
    name="Dara",
    age=20
)
```

Output:

Name: Dara
Age: 20
Course: Python

The order can differ because parameter names are supplied.

27. Positional vs Keyword Arguments

Positional:

```
student_info("Dara", 20, "Python")
```

Keyword:

```
student_info(  
    name="Dara",  
    age=20,  
    course="Python"  
)
```

Keyword arguments can improve clarity in function calls.

28. Arbitrary Positional Arguments — *args

Sometimes the number of arguments is unknown.

Use:

*args

Example:

```
def display_names(*names):  
    print(names)
```

```
display_names("Dara", "Sokha", "Vanna")
```

Output:

```
('Dara', 'Sokha', 'Vanna')
```

Inside the function, names behaves as a tuple.

29. Processing *args

Example:

```
def calculate_total(*numbers):  
    total = 0  
  
    for number in numbers:  
        total += number  
  
    return total
```

```
print(calculate_total(10, 20, 30))
```

Output:

60

30. Arbitrary Keyword Arguments — **kwargs

When an unknown number of keyword arguments is required, use:

****kwargs**

Example:

```
def student_info(**student):  
    print(student)
```

```
student_info(  
    name="Dara",  
    age=20,  
    course="Python"  
)
```

Output:

```
{'name': 'Dara', 'age': 20, 'course': 'Python'}
```

Inside the function, student behaves as a dictionary.

31. Accessing **kwargs Values

Example:

```
def student_info(**student):  
    print("Name:", student["name"])  
    print("Course:", student["course"])
```

```
student_info(  
    name="Dara",  
    course="Python"  
)
```

32. *args vs **kwargs

Feature	*args	**kwargs
Accepts	Arbitrary positional arguments	Arbitrary keyword arguments
Stored as	Tuple	Dictionary
Example	<code>func(1, 2, 3)</code>	<code>func(name="Dara")</code>

33. Variable Scope

Variable scope determines where a variable can be accessed.

The two basic scopes are:

1. Local
2. Global

34. Local Variables

A local variable is created inside a function.

Example:

```
def calculate():  
    total = 100  
    print(total)
```

```
calculate()
```

`total` is local to the function.

Normally, it cannot be accessed outside that function.

35. Global Variables

A global variable is defined outside functions.

Example:

```
course = "Python"
```

```
def display_course():  
    print(course)
```

```
display_course()
```

Output:

Python

The function can read the global variable.

36. Local and Global Variables Together

Example:

```
course = "Python"
```

```
def student():  
    name = "Dara"  
  
    print("Name:", name)  
    print("Course:", course)
```

```
student()
```

Here:

course → global variable

name → local variable

37. The global Keyword

The global keyword can be used when a function needs to assign to a global variable.

Example:

```
count = 0
```

```
def update_count():  
    global count  
    count = 5
```

```
update_count()
```

```
print(count)
```

Output:

5

For basic programs, relying excessively on global variables should generally be avoided when values can be passed through parameters and return values instead.

38. The pass Statement in Functions

Python does not allow an empty function body.

The pass statement can be used as a placeholder.

Example:

```
def future_function():  
    pass
```

The function is syntactically valid but performs no operation.

39. Why pass Is Useful

pass can be useful when:

- Planning program structure.
- Creating function placeholders.
- Developing one part of a program at a time.
- Defining functions that will be completed later.

Example:

```
def calculate_salary():  
    pass
```

40. Recursion

Recursion occurs when a function calls itself.

Example:

```
def countdown(number):  
    print(number)  
  
    if number > 1:  
        countdown(number - 1)
```

countdown(5)

Output:

```
5  
4  
3  
2  
1
```

41. Base Case in Recursion

A recursive function must have a stopping condition called a **base case**.

Example:

```
def countdown(number):  
    if number <= 0:  
        print("Done")  
        return  
  
    print(number)  
    countdown(number - 1)
```

countdown(3)

The condition:

```
if number <= 0:
```

prevents endless recursion.

42. Recursive Factorial

Factorial:

$5! = 5 \times 4 \times 3 \times 2 \times 1$

Recursive function:

```
def factorial(number):  
    if number <= 1:  
        return 1  
  
    return number * factorial(number - 1)  
  
print(factorial(5))
```

Output:

120

Execution concept:

```
factorial(5)  
↓  
5 × factorial(4)  
↓  
5 × 4 × factorial(3)  
↓
```

```
5 × 4 × 3 × factorial(2)
↓
5 × 4 × 3 × 2 × factorial(1)
↓
120
```

43. Recursion Limitations

Care is required with recursion.

Possible problems include:

- Missing base case.
- Incorrect base condition.
- Too many recursive calls.
- Higher memory use compared with some loops.
- Difficulty understanding complex recursive logic.

Use recursion where it makes the solution clearer and ensure there is a valid base case.

44. Lambda Functions

A lambda function is a short anonymous function.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x * x
```

```
print(square(5))
```

Output:

```
25
```

45. Lambda with Two Arguments

Example:

```
add = lambda a, b: a + b
```

```
print(add(5, 10))
```

Output:

46. Lambda with Conditional Expression

Example:

```
status = lambda score: "Pass" if score >= 50 else "Fail"

print(status(80))
print(status(40))
```

Output:

```
Pass
Fail
```

47. Regular Function vs Lambda

Regular function:

```
def square(number):
    return number * number
```

Lambda:

```
square = lambda number: number * number
```

Both can perform the same simple calculation.

Regular functions are usually more suitable for:

- Multiple statements.
- Complex logic.
- Reusable documented operations.

Lambda functions are useful for short expressions.

48. Functions and Code Reusability

Without a function:

```
a = 5
b = 10
print(a + b)
```

```
a = 20
b = 30
print(a + b)
```

Using a function:

```
def add(a, b):  
    return a + b  
  
print(add(5, 10))  
print(add(20, 30))
```

Functions reduce duplicate code.

49. Modular Programming

Modular programming divides a program into smaller functional units.

Example:

```
def display_menu():  
    print("1. Add")  
    print("2. Subtract")  
  
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b
```

Each function performs a specific task.

This improves:

- Organization
- Readability
- Reusability
- Testing
- Maintenance

50. Practical Function Example

```
def calculate_result(name, score=50):  
    status = "Pass" if score >= 50 else "Fail"  
  
    return "{}: {}".format(name, status)  
  
print(calculate_result("Dara", 80))  
print(calculate_result("Sokha", 40))
```

Output:

Dara: Pass
Sokha: Fail

51. Common Function Errors

Missing Colon

Incorrect:

```
def greet()  
    print("Hello")
```

Correct:

```
def greet():  
    print("Hello")
```

Incorrect Indentation

Incorrect:

```
def greet():  
print("Hello")
```

Correct:

```
def greet():  
    print("Hello")
```

Function Not Called

Example:

```
def greet():  
    print("Hello")
```

This defines the function but does not execute it.

Required:

```
greet()
```

Missing Required Argument

Example:

```
def greet(name):  
    print(name)
```

```
greet()
```

This produces a `TypeError` because the required `name` argument is missing.

Correct:

```
greet("Dara")
```

Too Many Arguments

Example:

```
def greet(name):  
    print(name)
```

```
greet("Dara", 20)
```

This produces a `TypeError` because the function accepts one positional argument but two are supplied.

return Outside a Function

Incorrect:

```
return 10
```

`return` must be used within a function.

Recursion without a Base Case

Incorrect:

```
def countdown(number):  
    print(number)  
    countdown(number - 1)
```

This repeatedly calls itself until Python's recursion limit is reached.

A valid stopping condition is required.

52. Debugging Functions

When a function does not work correctly:

1. Check `def`.
2. Check the function name.
3. Check parentheses.
4. Check the colon.
5. Check indentation.
6. Check parameter count.

7. Check argument count.
8. Check positional argument order.
9. Check keyword names.
10. Check default parameter rules.
11. Check return.
12. Check local/global variable usage.
13. Check *args and **kwargs.
14. Check recursive base cases.
15. Check lambda expressions.
16. Test functions with several inputs.
17. Compare expected and actual results.
18. Correct and execute again.

SELF-CHECK No. 11

Part A — Multiple Choice

1. Which keyword defines a function?

- A. function B. def C. func D. define

2. Which executes a function named greet?

- A. greet B. call greet C. greet() D. def greet()

3. A variable listed in a function definition is called a:

- A. Argument B. Parameter C. Keyword D. Module

4. A value supplied when calling a function is called an:

- A. Argument B. Parameter C. Function body D. Return

5. Which statement sends a result back from a function?

- A. print B. send C. return D. result

6. What is a default parameter?

- A. A parameter with a predefined value B. A global variable
C. A function call D. A list

7. Which is a keyword argument?

- A. student_info("Dara", 20) B. student_info(name="Dara")
C. student_info() D. def student_info()

8. Which syntax accepts arbitrary positional arguments?

- A. *args B. **args C. args* D. ...args

9. Which syntax accepts arbitrary keyword arguments?

- A. *kwargs B. **kwargs C. kwargs** D. #kwargs

10. What type is normally used to store values received by *args?

- A. Dictionary B. Tuple C. Set D. Integer

11. What type is normally used to store **kwargs?

- A. List B. Tuple C. Dictionary D. String

12. Which statement can provide an empty function body?

- A. continue B. break C. pass D. stop

13. What is recursion?

- A. A function calling itself B. A function with no parameters
C. A loop only D. A built-in function

14. A recursive function should normally contain a:

- A. Dictionary B. Base case
C. Global variable D. Lambda only

15. Which keyword creates a lambda function?

- A. lambda B. anonymous C. func D. short

ANSWER KEY No. 11.1

Part A

No.	Answer
1	B
2	C
3	B
4	A
5	C
6	A
7	B
8	A
9	B
10	B
11	C
12	C
13	A
14	B
15	A

OPERATION SHEET No. 11.1

Title: Create and Execute Python Functions

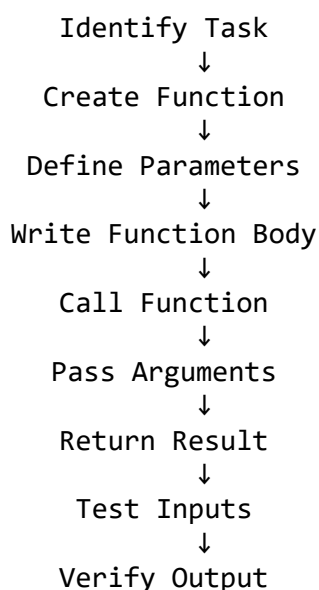
Performance Objective

Given a computer with Python installed and an approved Python development environment, the trainee must define functions, call functions, pass values and lists, apply parameters and arguments, return results, use default and keyword arguments, apply arbitrary arguments, create recursive and lambda functions, execute the program, and verify the output.

Description

This operation demonstrates the standard procedure for creating and using Python functions.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Guide	Copy	1
7	Python Function Exercise Files	Set	1
8	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
----------------	-----------------

Procedure/Step	Key Performance
1. Open Python IDE	Development environment opens successfully
2. Create .py file	Source file is created
3. Define function	Correct def syntax is used
4. Call function	Function executes correctly
5. Add parameters	Parameters match requirements
6. Pass arguments	Correct values are supplied
7. Pass a list	List is processed correctly
8. Use return	Required value is returned
9. Apply default parameter	Default value works correctly
10. Apply keyword argument	Named value is correctly matched
11. Apply *args	Multiple positional arguments are accepted
12. Apply **kwargs	Multiple named arguments are accepted
13. Apply pass	Placeholder function is valid
14. Create recursive function	Base case and recursive call are correct
15. Create lambda function	Expression executes correctly
16. Save and execute	Program runs successfully
17. Verify outputs	Results match requirements

Operation Procedure

Step 1 — Create and Call a Function

```
def greet():
    print("Hello Python")
```

```
greet()
```

Step 2 — Pass Values

```
def student_info(name, age):
    print(name, age)
```

```
student_info("Dara", 20)
```

Step 3 — Pass a List

```
def display_scores(scores):
    for score in scores:
        print(score)
```

```
display_scores([80, 90, 75])
```

Step 4 — Return a Value

```
def add(a, b):
    return a + b
```

```
print(add(5, 10))
```

Step 5 — Default Parameter

```
def welcome(name="Student"):
    print("Welcome", name)
```

```
welcome()
welcome("Dara")
```

Step 6 — Keyword Arguments

```
def profile(name, course):
    print(name, course)
```

```
profile(course="Python", name="Dara")
```

Step 7 — Arbitrary Arguments

```
def total(*numbers):
    return sum(numbers)
```

```
print(total(10, 20, 30))
```

Step 8 — Arbitrary Keyword Arguments

```
def profile_data(**data):
    print(data)
```

```
profile_data(name="Dara", age=20)
```

Step 9 — pass

```
def future_feature():
    pass
```

Step 10 — Recursion

```
def countdown(number):
    if number == 0:
        return

    print(number)
    countdown(number - 1)
```

```
countdown(3)
```

Step 11 — Lambda

```
square = lambda x: x * x
```

```
print(square(5))
```

Save as:

function_operations.py

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 11.1

Title: Create and Call Python Functions

Performance Task

Create user-defined functions and execute them using function calls.

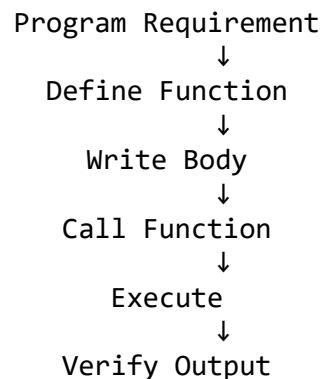
Performance Objective

Given Python IDLE/IDE and simple programming requirements, the trainee must define functions using def, apply proper indentation, and call each function correctly.

Description

The trainee practices basic function definition and execution.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Function Exercise File	Set	1
5	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Create function	def syntax is correct
2. Name function	Meaningful name is used
3. Apply colon	Function header is valid
4. Apply indentation	Function body is properly structured
5. Call function	Correct function name and parentheses are used
6. Call multiple times	Function is reusable
7. Verify output	Required text is displayed

Practical Exercise

Create:

```
def display_course():  
    print("AI Basic Python Developer")
```

display_course()

Create another function:

```
def welcome():  
    print("Welcome to Python Functions")
```

welcome()

welcome()

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

TASK SHEET No. 11.2

Title: Pass Values and Lists to Functions

Performance Task

Pass individual values and lists to Python functions.

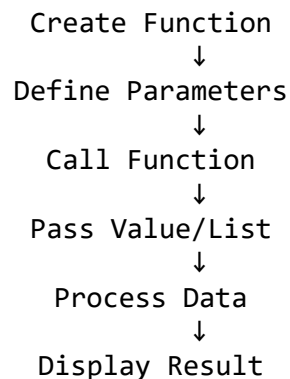
Performance Objective

Given function requirements, the trainee must define parameters, pass positional arguments, pass a list, and process list elements inside a function.

Description

The trainee practices parameters, arguments, and collection data.

Picture/Overview



Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Create parameters	Required parameter names are defined
2. Pass values	Correct positional arguments are supplied
3. Match arguments	Argument order is correct
4. Pass a list	List reaches the function correctly
5. Process list	All elements are processed
6. Verify results	Correct output is produced

Practical Exercise A

```
def student_info(name, age):  
    print("Name:", name)  
    print("Age:", age)
```

```
student_info("Dara", 20)
```

Practical Exercise B

```
def display_students(students):  
    for student in students:  
        print(student)
```

```
names = ["Dara", "Sokha", "Vanna"]
```

```
display_students(names)
```

Practical Exercise C

```
def display_scores(scores):  
    for score in scores:  
        print("Score:", score)
```

```
display_scores([80, 85, 90])
```

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 11.3

Title: Apply return, Default Parameters, and Keyword Arguments

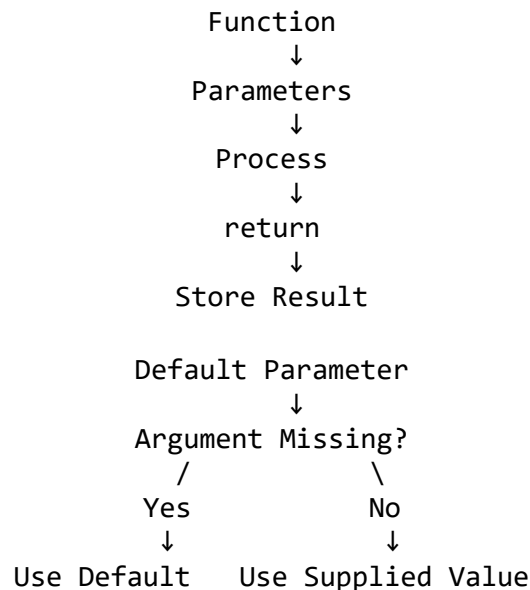
Performance Task

Develop functions that return results and accept default and keyword arguments.

Performance Objective

Given programming requirements, the trainee must correctly use return, create a default parameter, and supply arguments using parameter names.

Picture/Overview



Practical Exercise A — return

```
def add(a, b):  
    return a + b
```

```
result = add(5, 10)
```

```
print(result)
```

Expected:

15

Practical Exercise B — Default Value

```
def greet(name="Student"):  
    print("Hello", name)
```

```
greet()  
greet("Dara")
```

Practical Exercise C — Keyword Arguments

```
def student_info(name, age, course):  
    print("Name:", name)  
    print("Age:", age)  
    print("Course:", course)
```

```
student_info(  
    course="Python",  
    age=20,  
    name="Dara"  
)
```

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Define return function	Correct function structure is used
2. Apply return	Result is returned
3. Store result	Returned value is usable
4. Define default parameter	Default value is correctly positioned
5. Test omitted argument	Default value is used
6. Override default	Supplied value replaces default
7. Use keyword arguments	Correct parameter names are specified
8. Verify output	All results are correct

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

TASK SHEET No. 11.4

Title: Apply Arbitrary Arguments and pass

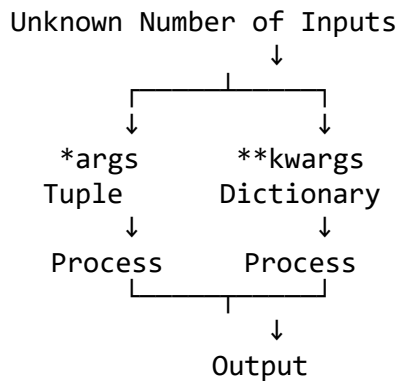
Performance Task

Develop functions using `*args`, `**kwargs`, and the `pass` statement.

Performance Objective

Given variable-length input requirements, the trainee must create functions accepting arbitrary positional and keyword arguments and create a placeholder function using `pass`.

Picture/Overview



Practical Exercise A — `*args`

```
def calculate_total(*scores):  
    total = 0  
  
    for score in scores:  
        total += score  
  
    return total
```

```
print(calculate_total(80, 90, 70))
```

Expected:

240

Practical Exercise B — `**kwargs`

```
def student_profile(**student):  
    print("Name:", student["name"])  
    print("Course:", student["course"])
```

```
student_profile(  
    name="Dara",  
    course="Python"  
)
```

Practical Exercise C — `pass`

```
def future_report():  
    pass
```

```
future_report()
```

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Apply *args	Arbitrary positional values are accepted
2. Process tuple data	All values are processed
3. Apply **kwargs	Arbitrary named values are accepted
4. Process dictionary data	Required keys are accessed
5. Apply pass	Empty function remains syntactically valid
6. Execute program	Program runs correctly
7. Verify output	Results meet requirements

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Output verification

TASK SHEET No. 11.5

Title: Implement Recursive Functions

Performance Task

Create and test recursive Python functions.

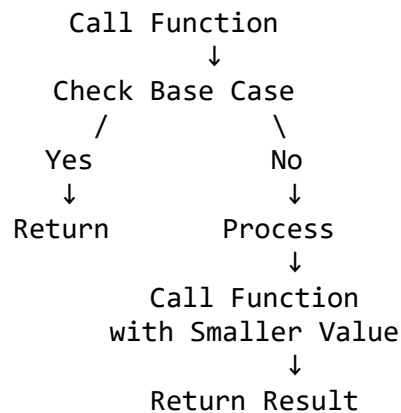
Performance Objective

Given repetitive mathematical or counting requirements, the trainee must define a recursive function with a valid base case and verify that recursive calls terminate correctly.

Description

The trainee practices self-calling functions.

Picture/Overview



Practical Exercise A — Countdown

```
def countdown(number):  
    if number <= 0:  
        print("Done")  
        return  
  
    print(number)  
  
    countdown(number - 1)
```

```
countdown(5)
```

Practical Exercise B — Factorial

```
def factorial(number):  
    if number <= 1:  
        return 1  
  
    return number * factorial(number - 1)
```

```
print(factorial(5))
```

Expected:

120

Procedure/Step | Key Performance

Procedure/Step	Key Performance
----------------	-----------------

Procedure/Step	Key Performance
1. Define recursive function	Function structure is correct
2. Create base case	Valid stopping condition is present
3. Make recursive call	Function calls itself correctly
4. Reduce problem	Argument approaches base case
5. Apply return	Recursive result is returned
6. Test function	Recursion terminates correctly
7. Verify output	Correct value is produced

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

TASK SHEET No. 11.6

Title: Implement Lambda Functions

Performance Task

Create and execute lambda functions for simple expressions.

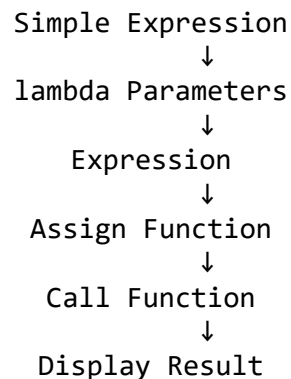
Performance Objective

Given simple calculation and decision requirements, the trainee must create lambda functions with one or more parameters and verify their results.

Description

The trainee practices anonymous functions for short expressions.

Picture/Overview



Practical Exercise A — Square

```
square = lambda number: number * number
```

```
print(square(5))
```

Expected:

25

Practical Exercise B — Addition

```
add = lambda a, b: a + b
```

```
print(add(5, 10))
```

Expected:

15

Practical Exercise C — Pass/Fail

```
status = lambda score: "Pass" if score >= 50 else "Fail"
```

```
print(status(80))
```

```
print(status(40))
```

Expected:

Pass

Fail

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Apply lambda keyword	Correct syntax is used

Procedure/Step	Key Performance
2. Define parameter	Parameter receives value
3. Write expression	Valid single expression is used
4. Assign lambda	Function reference is created
5. Call lambda	Correct arguments are supplied
6. Verify results	Expected values are produced

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

JOB SHEET No. 11.1

Title: Develop a Student Result Processing Program Using Functions

Performance Task

Develop a complete modular Python program that uses multiple function techniques to process student information and results.

Performance Objective

Given a computer, Python development environment, student data, and programming requirements, the trainee must create and call functions, pass values and lists, use return statements, default parameters, keyword arguments, arbitrary arguments, pass, recursion, and lambda functions, test the program, debug errors, and verify the output.

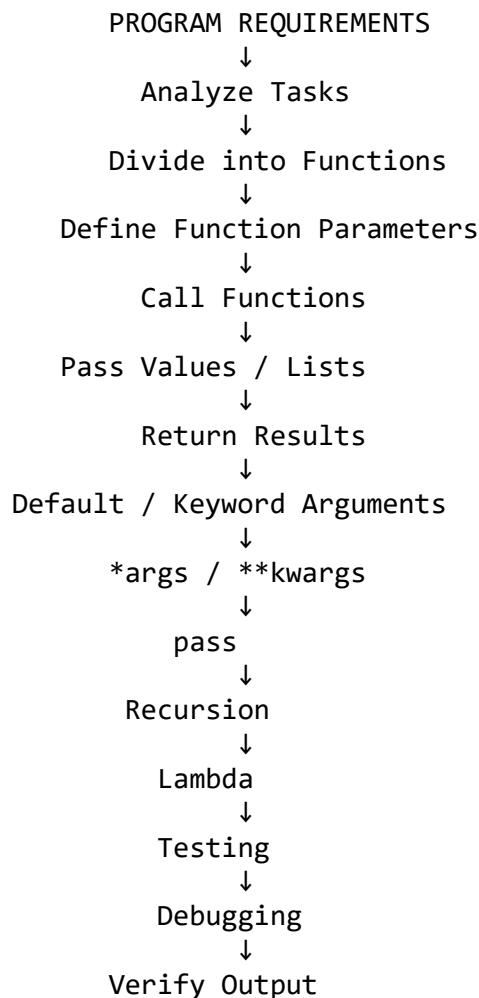
Description

Develop:

`student_function_manager.py`

The program must demonstrate the principal competencies required in LO11.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Python Function Exercise Files	Set	1
6	Python Programming Reference Guide	Copy	1
7	Programming Worksheet	Copy	1
8	USB Flash Drive	Unit	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

Procedure/Step	Key Performance
1. Analyze program requirements	Required functions are identified
2. Divide program into functions	Each function has a clear task
3. Create functions using def	Correct syntax is applied
4. Call functions	Functions execute correctly
5. Define parameters	Appropriate parameters are used
6. Pass positional arguments	Values match correct parameters
7. Pass a list	List data is processed correctly
8. Apply return	Results are returned correctly
9. Apply default parameters	Default behavior is correct
10. Apply keyword arguments	Named arguments are correctly matched
11. Apply *args	Arbitrary positional data is processed
12. Apply **kwargs	Arbitrary keyword data is processed
13. Apply pass	Placeholder function is valid
14. Create recursive function	Base case and recursive call are correct
15. Create lambda function	Simple expression is correct
16. Apply local variables	Scope is appropriate
17. Save source file	.py extension is used
18. Execute program	Program runs successfully
19. Test functions independently	Results are checked
20. Identify errors	Function-related errors are recognized
21. Debug program	Errors are corrected
22. Verify final output	Results match requirements

Program Requirements

The program must include:

1. At least five user-defined functions.
2. Function calls.

3. Parameters and arguments.
4. A function receiving a list.
5. A function returning a value.
6. A default parameter.
7. Keyword arguments.
8. `*args`.
9. `**kwargs`.
10. A placeholder function using `pass`.
11. A recursive function.
12. A lambda function.
13. Local variables.
14. Program testing.

Part 1 — Display Student Information

Create:

```
def display_student(name, course="Python"):
    print("Name:", name)
    print("Course:", course)
```

Call using a keyword:

```
display_student(
    course="AI Basic Python Developer",
    name="Dara"
)
```

Part 2 — Calculate Total Score

Create:

```
def calculate_total(scores):
    total = 0

    for score in scores:
        total += score

    return total
```

Use:

```
scores = [80, 85, 90]
```

```
total = calculate_total(scores)
```

```
print("Total:", total)
```

Expected:

255

Part 3 — Calculate Average

```
def calculate_average(scores):
    total = calculate_total(scores)

    return total / len(scores)
```

Expected:
85.0

Part 4 — Arbitrary Scores with *args

```
def score_total(*scores):  
    return sum(scores)
```

```
print(score_total(80, 85, 90))
```

Part 5 — Student Information with **kwargs

```
def show_profile(**student):  
    print("Student Profile")  
  
    for key, value in student.items():  
        print(key, ":", value)
```

```
show_profile(  
    name="Dara",  
    level="II",  
    course="AI Basic Python Developer"  
)
```

Part 6 — Placeholder Function

```
def export_report():  
    pass
```

This function represents future development.

Part 7 — Recursive Function

Create factorial:

```
def factorial(number):  
    if number <= 1:  
        return 1  
  
    return number * factorial(number - 1)
```

Test:

```
print("Factorial of 5:", factorial(5))
```

Expected:

Factorial of 5: 120

Part 8 — Lambda Function

Create pass/fail check:

```
result_status = lambda score: "Pass" if score >= 50 else "Fail"
```

Test:

```
print(result_status(85))
```

Expected:

Pass

Example Complete Program

Student Function Management Program

```
def display_student(name, course="Python"):
    print("Name:", name)
    print("Course:", course)

def calculate_total(scores):
    total = 0

    for score in scores:
        total += score

    return total

def calculate_average(scores):
    total = calculate_total(scores)

    return total / len(scores)

def score_total(*scores):
    return sum(scores)

def show_profile(**student):
    for key, value in student.items():
        print(key, ":", value)

def export_report():
    pass

def factorial(number):
    if number <= 1:
        return 1

    return number * factorial(number - 1)

result_status = lambda score: "Pass" if score >= 50 else "Fail"
```

```

print("=====")
print("          STUDENT FUNCTION MANAGER")
print("=====")

# Function call with keyword arguments
display_student(
    course="AI Basic Python Developer",
    name="Dara"
)

# Pass List
scores = [80, 85, 90]

print("\nSCORES")
print(scores)

# Return values
total = calculate_total(scores)
average = calculate_average(scores)

print("Total:", total)
print("Average:", average)

# Arbitrary positional arguments
print("\nARBITRARY ARGUMENTS")
print("Total:", score_total(80, 85, 90))

# Arbitrary keyword arguments
print("\nSTUDENT PROFILE")
show_profile(
    name="Dara",
    level="II",
    course="AI Basic Python Developer"
)

# Placeholder function
export_report()

# Recursion
print("\nRECURSION")
print("Factorial of 5:", factorial(5))

# Lambda
print("\nLAMBDA")
print("Score 85:", result_status(85))
print("Score 40:", result_status(40))

```

Expected Main Output

```
=====
STUDENT FUNCTION MANAGER
=====
```

```
Name: Dara
Course: AI Basic Python Developer
```

```
SCORES
[80, 85, 90]
Total: 255
Average: 85.0
```

```
ARBITRARY ARGUMENTS
Total: 255
```

```
STUDENT PROFILE
name : Dara
level : II
course : AI Basic Python Developer
```

```
RECURSION
Factorial of 5: 120
```

```
LAMBDA
Score 85: Pass
Score 40: Fail
```

Testing Requirements

Test Item	Expected Result	Actual Result	Pass/Fail
Function call works	Yes	_____	_____
Default course if omitted	Python	_____	_____
calculate_total([80,85,90])	255	_____	_____
Average	85.0	_____	_____
score_total(80,85,90)	255	_____	_____
**kwargs name	Dara	_____	_____
Placeholder function runs	No error	_____	_____
factorial(5)	120	_____	_____
Lambda with 85	Pass	_____	_____
Lambda with 40	Fail	_____	_____

Debugging Requirements

Error 1 — Missing Colon

```
Incorrect:
def greet()
    print("Hello")
```

Correct:

```
def greet():  
    print("Hello")
```

Error 2 — Missing Argument

Incorrect:

```
def greet(name):  
    print(name)
```

`greet()`

Correct:

`greet("Dara")`

Error 3 — Incorrect Default Parameter Order

Incorrect:

```
def student(course="Python", name):  
    print(name, course)
```

Correct:

```
def student(name, course="Python"):  
    print(name, course)
```

Error 4 — Missing Recursive Base Case

Incorrect:

```
def countdown(number):  
    print(number)  
    countdown(number - 1)
```

Correct:

```
def countdown(number):  
    if number <= 0:  
        return  
  
    print(number)  
    countdown(number - 1)
```

WORKSHEET No. 11.1

Title: Python Function Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — Function Structure

Complete:

```
_____ greet():  
    print("Hello")
```

Answers: _____

Part B — Identify Parameter and Argument

Given:

```
def greet(name):  
    print(name)
```

```
greet("Dara")
```

Parameter: _____

Argument: _____

Part C — Function Call

Write a call for:

```
def display_course():  
    print("Python")
```

Answer: _____

Part D — Passing Values

Complete:

```
def add(a, b):  
    print(a + b)
```

```
add(_____, _____)
```

Use values 5 and 10.

Part E — Pass a List

Given:

```
scores = [80, 90, 75]
```

Complete:

```
def display_scores(scores):  
    for score in scores:  
        print(score)
```

```
display_scores(_____)
```

Part F — return

Complete:

```
def multiply(a, b):  
    _____ a * b
```

Answer:

Part G — Default Parameter

Complete:

```
def greet(name=_____):  
    print("Hello", name)
```

Use:

Student

Part H — Keyword Arguments

Given:

```
def student(name, course):  
    print(name, course)
```

Write a function call using keyword arguments:

Part I — *args

Complete:

```
def total(_____ numbers):  
    return sum(numbers)
```

Answer:

Part J — **kwargs

Complete:

```
def profile(_____ student):  
    print(student)
```

Answer:

Part K — pass

Complete:

```
def future_function():  
    _____
```

Part L — Recursion

What important component is missing?

```
def countdown(number):  
    print(number)  
    countdown(number - 1)
```

Answer:

Part M — Lambda

Complete:

square = _____ x: x * x

Answer:

What is:

square(5)

Answer:

Part N — Local and Global Variables

Given:

course = "Python"

```
def student():  
    name = "Dara"  
    print(name)  
    print(course)
```

Global variable: _____

Local variable: _____

PERFORMANCE CHECKLIST No. 11.1

Title: Develop Program Using Functions

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Explained the purpose of Python functions	☐	☐
2	Identified built-in functions	☐	☐
3	Identified user-defined functions	☐	☐
4	Identified recursive functions	☐	☐
5	Identified lambda functions	☐	☐
6	Created a function using def	☐	☐
7	Applied correct function naming conventions	☐	☐
8	Applied correct indentation	☐	☐
9	Called a function correctly	☐	☐
10	Called a function multiple times where required	☐	☐
11	Distinguished parameters from arguments	☐	☐
12	Passed a single value into a function	☐	☐
13	Passed multiple values into a function	☐	☐
14	Passed a list into a function	☐	☐
15	Processed list data inside a function	☐	☐
16	Applied positional arguments correctly	☐	☐
17	Applied return correctly	☐	☐
18	Stored or used a returned result	☐	☐
19	Applied a default parameter value	☐	☐
20	Overrode a default parameter where required	☐	☐
21	Applied keyword arguments correctly	☐	☐
22	Applied *args correctly	☐	☐
23	Processed arbitrary positional arguments	☐	☐
24	Applied **kwargs correctly	☐	☐
25	Processed arbitrary keyword arguments	☐	☐
26	Identified local variable scope	☐	☐
27	Identified global variable scope	☐	☐
28	Applied global only where appropriate	☐	☐
29	Applied pass in a function	☐	☐
30	Created a recursive function	☐	☐
31	Defined a valid recursion base case	☐	☐
32	Ensured recursive calls approach the base case	☐	☐
33	Returned a recursive result where required	☐	☐
34	Created a lambda function	☐	☐
35	Applied one or more lambda parameters correctly	☐	☐

No.	Criteria	Yes	No
36	Applied lambda to a simple expression	☐	☐
37	Applied functions to improve code reusability	☐	☐
38	Divided a program into functional modules	☐	☐
39	Saved program with .py extension	☐	☐
40	Executed the function program successfully	☐	☐
41	Tested functions with different inputs	☐	☐
42	Tested default parameter behavior	☐	☐
43	Tested keyword arguments	☐	☐
44	Tested *args	☐	☐
45	Tested **kwargs	☐	☐
46	Tested recursive function termination	☐	☐
47	Tested lambda functions	☐	☐
48	Identified function syntax errors	☐	☐
49	Identified parameter/argument errors	☐	☐
50	Identified recursion errors	☐	☐
51	Applied appropriate debugging techniques	☐	☐
52	Corrected identified errors	☐	☐
53	Verified final program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____